

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import
StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import
silhouette_score
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style("whitegrid")
plt.rcParams["figure.dpi"] = 150

df =
pd.read_csv("/home/claude/proj/custom
er_data.csv")

# --- Feature engineering: RFM + a
couple of behavioral signals ---
features = ["RecencyDays",
"Frequency", "MonetaryTotal",
"DiscountSensitivity",
"OnlinePurchaseRatio"]
X = df[features].copy()

scaler = StandardScaler()
```

```

X_scaled = scaler.fit_transform(X)

# --- Determine optimal k via elbow +
silhouette ---
inertias, sil_scores = [], []
K_range = range(2, 9)
for k in K_range:
    km = KMeans(n_clusters=k,
random_state=42, n_init=10)
    labels = km.fit_predict(X_scaled)
    inertias.append(km.inertia_)

sil_scores.append(silhouette_score(X_
scaled, labels))

fig, ax1 = plt.subplots(figsize=(7,
4.5))
ax1.plot(list(K_range), inertias,
marker="o", color="#2E5EAA")
ax1.set_xlabel("Number of Clusters
(k)")
ax1.set_ylabel("Inertia",
color="#2E5EAA")
ax2 = ax1.twinx()
ax2.plot(list(K_range), sil_scores,
marker="s", color="#D97B29")
ax2.set_ylabel("Silhouette Score",
color="#D97B29")
plt.title("Optimal Cluster Selection:

```

```

Elbow & Silhouette")
fig.tight_layout()
plt.savefig("/home/claude/proj/elbow_
silhouette.png")
plt.close()

best_k = 5
km = KMeans(n_clusters=best_k,
random_state=42, n_init=10)
df["Segment"] =
km.fit_predict(X_scaled)

# --- Label segments by their
characteristics ---
profile = df.groupby("Segment")
[features].mean()

def label_segment(row):
    if row["MonetaryTotal"] >
profile["MonetaryTotal"].quantile(0.7
5) and row["RecencyDays"] <
profile["RecencyDays"].median():
        return "Champions"
    if row["RecencyDays"] >
profile["RecencyDays"].quantile(0.7):
        return "At Risk"
    if row["Frequency"] <
profile["Frequency"].quantile(0.3):
        return "New / Occasional"

```

```

        if row["DiscountSensitivity"] >
profile["DiscountSensitivity"].quanti
le(0.65):
            return "Bargain Hunters"
        return "Loyal Regulars"

labels_map = {seg:
label_segment(profile.loc[seg]) for
seg in profile.index}
df["SegmentName"] =
df["Segment"].map(labels_map)

df.to_csv("/home/claude/proj/customer
_data_segmented.csv", index=False)

# --- Segment profile summary table -
--
summary =
df.groupby("SegmentName").agg(
    Customers=("CustomerID",
"count"),
    AvgAge=("Age", "mean"),
    AvgIncome=("AnnualIncome",
"mean"),
    AvgRecencyDays=("RecencyDays",
"mean"),
    AvgFrequency=("Frequency",
"mean"),
    AvgMonetary=("MonetaryTotal",

```

```

"mean"),
    AvgDiscountSensitivity=
("DiscountSensitivity", "mean"),
    AvgOnlineRatio=
("OnlinePurchaseRatio", "mean"),
).round(1).sort_values("AvgMonetary",
ascending=False)
summary["PctOfBase"] =
(summary["Customers"] / len(df) *
100).round(1)
summary.to_csv("/home/claude/proj/seg
ment_summary.csv")
print(summary)

# --- Visual 1: Scatter of segments
(Frequency vs Monetary) ---
plt.figure(figsize=(7.5, 5.5))
palette = sns.color_palette("Set2",
df["SegmentName"].nunique())
sns.scatterplot(data=df,
x="Frequency", y="MonetaryTotal",
hue="SegmentName", palette=palette,
s=45, alpha=0.8)
plt.title("Customer Segments:
Purchase Frequency vs. Total Spend")
plt.xlabel("Purchase Frequency
(orders)")
plt.ylabel("Total Monetary Value
($)")

```

```
plt.legend(title="Segment",
bbox_to_anchor=(1.02, 1), loc="upper
left")
plt.tight_layout()
plt.savefig("/home/claude/proj/segmen
t_scatter.png")
plt.close()
```

```
# --- Visual 2: Segment size ---
plt.figure(figsize=(6.5, 4.5))
counts =
df["SegmentName"].value_counts()
sns.barplot(x=counts.values,
y=counts.index, palette="Set2")
plt.title("Customer Count by
Segment")
plt.xlabel("Number of Customers")
plt.ylabel("")
plt.tight_layout()
plt.savefig("/home/claude/proj/segmen
t_counts.png")
plt.close()
```

```
# --- Visual 3: RFM heatmap of
segment averages (normalized) ---
heat_features = ["RecencyDays",
"Frequency", "MonetaryTotal",
"DiscountSensitivity",
"OnlinePurchaseRatio"]
```

```

heat_data = df.groupby("SegmentName")
[heat_features].mean()
heat_norm = (heat_data -
heat_data.min()) / (heat_data.max() -
heat_data.min())
plt.figure(figsize=(7.5, 4.5))
sns.heatmap(heat_norm,
annot=heat_data.round(1), fmt="",
cmap="YlGnBu", cbar_kws={"label":
"Relative Intensity"})
plt.title("Segment Characteristic
Profile (normalized)")
plt.tight_layout()
plt.savefig("/home/claude/proj/segmen
t_heatmap.png")
plt.close()

# --- Visual 4: Demographics by
segment ---
fig, axes = plt.subplots(1, 2,
figsize=(11, 4.5))
sns.boxplot(data=df, x="SegmentName",
y="Age", ax=axes[0], palette="Set2")
axes[0].set_title("Age Distribution
by Segment")
axes[0].tick_params(axis="x",
rotation=30)
sns.boxplot(data=df, x="SegmentName",
y="AnnualIncome", ax=axes[1],

```

```

palette="Set2")
axes[1].set_title("Income
Distribution by Segment")
axes[1].tick_params(axis="x",
rotation=30)
plt.tight_layout()
plt.savefig("/home/claude/proj/segment_demographics.png")
plt.close()

# --- Visual 5: Preferred category
mix by segment ---
cat_mix =
pd.crosstab(df["SegmentName"],
df["PreferredCategory"],
normalize="index") * 100
plt.figure(figsize=(8.5, 5))
cat_mix.plot(kind="bar",
stacked=True, colormap="tab20",
ax=plt.gca())
plt.title("Preferred Category Mix by
Segment (%)")
plt.ylabel("% of Segment")
plt.xticks(rotation=30, ha="right")
plt.legend(bbox_to_anchor=(1.02, 1),
loc="upper left", fontsize=8)
plt.tight_layout()
plt.savefig("/home/claude/proj/segment_categories.png")

```



```
plt.close()
```

```
print("\nDone. Files written to  
/home/claude/proj/")
```